

Statistical and Machine Learning SS26

Exercise Sheet 1

Exercise 1: My First R Script

Open a new document and save it as `myfirstfile.R` in a new folder called `SML_SS26`. Answer the following questions in this file and indicate exercise numbers in comments.

- What is your current working directory?
- Set your working directory to `SML_SS26`.

Note: Save all your solutions for the following exercises in `myfirstfile.R`. Use comments to structure your document!

Exercise 2: Object Assignment

- Assign the value 42 to an object named `x` and the value 8 to an object named `y`. Compute and print the sum, difference, product, and quotient of `x` and `y`.
- Assign your first name as a character string to an object called `my_name`. Print the object. What does `mode(my_name)` return?
- What will happen when running line 3 of the code below? Verify your answer in R:

```
n1 <- 100
N2 <- 200
n1 + n2
```

Exercise 3: Data Types

R has five core data types. For each of the objects below:

- determine the data type
- explain in a comment what type it is and why

```
a <- 3.14
b <- 7L
c <- "Statistics"
d <- TRUE
e <- factor(c("low", "medium", "high", "medium", "low"))
```

- a) What is the difference between `b <- 7L` and `b2 <- 7`? Check with `is.integer()` and `is.numeric()`
- b) Inspect the factor `e`:
- How many levels does it have?
 - How many observations?
 - What happens when you apply `as.integer(e)`? Explain the result
-

Exercise 4: Vectors

- a) Create the following vectors:
- A numeric vector `v1` containing the values 4, 8, 15, 16, 23, 42
 - A character vector `v2` with the names of four cities of your choice
 - A logical vector `v3` of length 5
- b) Using `v1`, compute:
- the sum, mean, median, variance, and standard deviation
 - the minimum and maximum values
- c) **Sequences and repetition:** Create the following vectors without typing each element manually:
- A vector of integers from 1 to 20
 - A vector of values from 0 to 1 in steps of 0.1
 - A vector that repeats the pattern `c(1, 2, 3)` four times (use `rep()`)
- d) **Vectorised operations:** R applies arithmetic operations element-wise. Predict the output before running:

```
x <- c(1, 2, 3, 4)
y <- c(10, 20, 30, 40)
x + y
x * y
x^2
```

Exercise 5: Indexing and Subsetting Vectors

Use the vector `v1 <- c(4, 8, 15, 16, 23, 42)` for this exercise

- a) Extract the third element of `v1`
- b) Extract all elements **except** the second
- c) Extract all elements that are greater than 15
- d) Replace the fourth element with the value 99
- e) What does `v1[c(1, 3, 5)]` return?
-

Exercise 6: Matrices

- a) Create a 4×3 matrix **M** filled with integers from 1 to 12. Check whether it is filled by column or by row. Then create the same matrix filled by the opposite.
 - b) Assign the following column names to **M**: "A", "B", "C". Assign row names "r1", "r2", "r3", "r4".
 - c) Access:
 - The element in row 3, column 2.
 - The entire second row.
 - The entire third column.
 - The sub-matrix formed by rows 1–2 and columns 2–3.
 - d) Use `apply()` to compute:
 - The **row sums** of **M**.
 - The **column means** of **M**.
 - e) Create two 2×2 matrices **A** and **B** of your choice. Compute $(A * B)$ and $(A \%*\% B)$. What is the difference?
-

Exercise 7: Arrays

The built-in R object `UCBAdmissions` is a 3-dimensional array with the number of admitted and rejected students at Berkeley University in 1973. Compute:

- a) the number of students admitted and rejected
- b) the number of males and females
- c) the number of students by departments
- d) the number of students admitted and rejected by gender
- e) the number of males and females by departments

Use the function `apply()` to solve this task

Exercise 8: Lists

- a) Create a list **student** with the following named elements:
 - **name**: your first name (character)
 - **age**: your age (numeric)
 - **courses**: a character vector with two course names
 - **passed**: a logical value
- b) Access the **courses** element using three different methods:
 - single square brackets `[ ]`
 - double square brackets `[[ ]]`

- the `$` operator

What is the difference in the type of the returned object? Use `class()` to check

- Add a new element `grade` with a numeric value to the list using `$`
 - Use `str()` to inspect the full structure of `student`
-

Exercise 9: Data Frames

- Create a data frame `students_df` with the following four columns (at least 5 rows of your choice):
 - `name` (character)
 - `age` (numeric)
 - `grade` (numeric, values between 1.0 and 5.0)
 - `passed` (logical: TRUE if `grade <= 4.0`)
 - Inspect the data frame.
 - Access:
 - The `grade` column using two different commands
 - The third row.
 - The element in row 2, column 3.
 - Add a new column `grade_letter` that categorizes grades: "A" for grade ≤ 1.5 , "B" for grade ≤ 2.5 , "C" for grade ≤ 3.5 , "D" for grade ≤ 4.0 , "F" otherwise.
 - Load the built-in `iris` dataset with `data(iris)`.
 - How many rows and columns does it have?
 - What are the column names?
 - How many observations are there per species?
 - Subset the data frame to only include rows where `Sepal.Length > 5` and the `Species` is "setosa".
-

Exercise 10: Functions

- Write a function `celsius_to_fahrenheit(temp_c)` that converts a temperature from Celsius to Fahrenheit using the formula $F = \frac{9}{5} \cdot C + 32$. Test it with the values 0, 100, and -40.
 - Write a function `my_summary(x)` that takes a numeric vector `x` and returns a **named list** containing:
 - `mean`, `sd`, `min`, `max`, and `n` (length of the vector).

Test it on `v1 <- c(4, 8, 15, 16, 23, 42)` and `v2 <- c(4, 8, 15, 16, 23, 42, NA)`
 - Extend `my_summary()` by adding an optional argument `na.rm = FALSE` that is passed on to `mean()`, `sd()`, etc. Test it on `c(1, 2, NA, 4, 5)` What does `na.rm = FALSE / TRUE` do?
-

Exercise 11: Control Flow

- Write an **if-else** statement that prints "Pass" if a grade value is ≤ 4.0 and "Fail" otherwise. Test it for grades 2.3 and 4.7.
 - Use **ifelse()** (vectorised) to apply the same pass/fail logic to the **grade** column of **students_df**.
 - Write a **for** loop that prints the square of each integer from 1 to 10.
 - Write a **while** loop that starts at 1 and keeps doubling the value until it exceeds 1000. Print the final value.
 - Use **sapply()** to apply your **celsius_to_fahrenheit()** function (from Exercise 9a) to the vector **c(-20, 0, 20, 37, 100)**. Do the same thing with a **for** loop - which solution do you prefer and why?
-

Exercise 12: Reading and Writing Data

- Download the file **dax_prices.csv** from the HU Box folder and save it to your current working directory. Read the file **dax_prices.csv** into a data frame **dax.prices**.
 - Inspect the first rows of **dax.prices**. Are the prices (contained in variable **DAX**) correctly interpreted as numeric?
 - Rename the column **DAX** to **DAX Prices**.
 - Write **dax.prices** to a file named **dax_prices.txt** in your working directory, with ";" as separator and "." as decimal point.
 - Read **dax_prices.txt** into a data frame **dax.prices.txt**. Make sure the prices are correctly interpreted as numeric
-

Exercise 13: Linear Regression

- Load the built-in **swiss** dataset with **data(swiss)** and inspect it. How many observations and variables does it contain?
 - Fit a simple linear regression model predicting **Fertility** from **Education** only:
 - Is the coefficient for **Education** statistically significant?
 - Interpret the coefficient: what does it mean substantively?
 - Extend the model by adding **Infant.Mortality** as a second predictor:
 - How does the R^2 change compared to the simple model?
 - Extract the fitted values and residuals using **fitted()** and **resid()**.
 - Use **coef()** to extract the estimated coefficients and **vcov()** to extract the variance-covariance matrix of the estimates. What does the diagonal of **vcov(fit_multi)** represent?
 - Add an interaction term between **Education** and **Infant.Mortality** to the model. Write out the model formula using R's formula interface and interpret whether the interaction is significant.
-